ARYA COLLEGE OF ENGINEERING · AGENTIC AI INDUSTRIAL TRAINING

Practical Guide — Part 2

Lectures 15 & 16 — Token Optimization, Caching & Capstone Project

Sections 7–12: Chunking · Cost Control · Prompt Caching · Rate Limiting · Full AI Study Assistant

4.2 Reducing Token Usage · 4.3 Cost Estimation · 4.4 Prompt Caching · 4.5 Rate Limiting
Capstone: AI Study Assistant Chatbot · Full Production-Ready System

Section 7 (4.2): Reducing Token Usage & Chunking

Save 60–80% on API costs with smart prompt engineering

Why Token Reduction Matters

Strategy	Tokens Saved	When to Use
Remove filler phrases	**10–20%**	*Always — 'Please could you kindly...' → 'Explain:'*
Use bullet instructions	**15–25%**	*Complex multi-step tasks*
Truncate long context	**30–60%**	*When pasting big documents*
Chunking large inputs	**40–70%**	*PDFs, books, long codebases*
Few-shot → Zero-shot	**5–15%**	*When model already knows the task*
Cache repeated prompts	**60–90%**	*Shared system messages across users*

Python Concepts Used: `split()`, `len()`, `list comprehension`, `dict`, `enumerate()`, `math.ceil`, `generator`

```
# ── chunking_demo.py ──────────────────────────────────────────
# Process long documents without hitting token limits
# Real use: summarize a 50-page PDF, analyze a full codebase
```

```python
import google.generativeai as genai
import os, math, time
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
model = genai.GenerativeModel('gemini-1.5-flash')

# ── Chunk a long text into pieces ──────────────────────────────
def chunk_text(text: str, max_words: int = 300) -> list[str]:
    '''
    Split text into chunks of ~max_words words.
    Uses list comprehension + range() — very Pythonic.
    '''
    words  = text.split()                        # split on whitespace
    chunks = []
    # range(start, stop, step) — jumps max_words at a time
    for i in range(0, len(words), max_words):
        chunk = words[i : i + max_words]         # slice the list
        chunks.append(' '.join(chunk))           # join back to string
    return chunks

def summarize_chunk(chunk: str, chunk_num: int, total: int) -> str:
    '''Summarize one chunk with context awareness.'''
    prompt = f'''You are summarizing part {chunk_num} of {total} of a document.
    Give a 2-sentence summary only. Be concise.

    TEXT:
    {chunk}

    SUMMARY:'''
    # Notice: prompt is SHORT and specific — no wasted words
    config   = genai.GenerationConfig(max_output_tokens=100, temperature=0.3)
    response = model.generate_content(prompt, generation_config=config)
    return response.text.strip()

def summarize_long_document(text: str, chunk_size: int = 300) -> str:
    '''
    Full pipeline: chunk → summarize each → combine → final summary.
    Map-Reduce pattern: split work, do it in parallel-style, combine.
    '''
    chunks         = chunk_text(text, max_words=chunk_size)
    total          = len(chunks)
    chunk_summaries = []

    print(f'Document split into {total} chunks of ~{chunk_size} words each')
    print(f'Estimated tokens: {total * chunk_size // 3:,} input tokens')
    print()

    # Process each chunk (enumerate gives index + value)
    for i, chunk in enumerate(chunks, start=1):    # enumerate starts at 1
        print(f'  Processing chunk {i}/{total}...', end='', flush=True)
        summary = summarize_chunk(chunk, i, total)
        chunk_summaries.append(f'Part {i}: {summary}')
        print(' done')
        time.sleep(0.5)   # gentle rate limiting

    # Final merge: combine all chunk summaries into one
    all_summaries = '\n'.join(chunk_summaries)    # join list with newlines
    merge_prompt  = f'''Here are summaries of each part of a document.
    Write ONE final summary paragraph (4-5 sentences) combining everything.
```

```
    {all_summaries}

    FINAL SUMMARY:'''

    config   = genai.GenerationConfig(max_output_tokens=200, temperature=0.5)
    response = model.generate_content(merge_prompt, generation_config=config)
    return response.text.strip()

# ── Bonus: Prompt compression helper ──────────────────────────────
def compress_prompt(verbose_prompt: str) -> str:
    '''
    Before sending to AI, strip filler words to save tokens.
    Demonstrates: str methods, list comprehension, filter.
    '''
    filler_phrases = [
        'please', 'could you', 'kindly', 'i would like you to',
        'can you', 'would you mind', 'i was wondering if',
        'as you may know,', 'it is important to note that',
    ]
    compressed = verbose_prompt.lower()
    for phrase in filler_phrases:
        compressed = compressed.replace(phrase, '')  # str.replace()
    # Remove double spaces left behind
    while '  ' in compressed:
        compressed = compressed.replace('  ', ' ')
    return compressed.strip().capitalize()

# ── Demo ──────────────────────────────────────────────────────────
if __name__ == '__main__':
    # Simulate a long document (in real use: open('textbook.txt').read())
    sample_text = '''
    Python is a high-level, interpreted programming language known for its
    clear syntax and readability. Guido van Rossum created Python in 1991.
    It supports multiple programming paradigms including procedural,
    object-oriented, and functional programming. Python has a large standard
    library and a huge ecosystem of third-party packages available via pip.
    The language is widely used in web development, data science, artificial
    intelligence, automation, and scientific computing. Django and Flask are
    popular web frameworks. NumPy, Pandas, and Scikit-learn power data science.
    TensorFlow and PyTorch are used for deep learning. Python's simplicity
    makes it ideal for beginners while remaining powerful for experts.
    ''' * 5  # multiply string to make it longer for demo

    print('=== CHUNKING DEMO ===')
    final = summarize_long_document(sample_text, chunk_size=80)
    print(f'\nFINAL SUMMARY:')
    print(final)

    print('\n=== PROMPT COMPRESSION DEMO ===')
    verbose = 'Could you please kindly explain what a Python decorator is?'
    compressed = compress_prompt(verbose)
    print(f'Before ({len(verbose.split())} words): {verbose}')
    print(f'After  ({len(compressed.split())} words): {compressed}')
```

Output:

```
=== CHUNKING DEMO ===
Document split into 3 chunks of ~80 words each
Estimated tokens: 150 input tokens

  Processing chunk 1/3... done
```

```
  Processing chunk 2/3... done
  Processing chunk 3/3... done

FINAL SUMMARY:
Python is a versatile, high-level programming language created in 1991,
celebrated for its readable syntax and multi-paradigm support. It powers
web development (Django, Flask), data science (NumPy, Pandas), and AI
(TensorFlow, PyTorch). Its beginner-friendly design and vast ecosystem
make it the world's most popular language for both learning and production.

=== PROMPT COMPRESSION DEMO ===
Before (12 words): Could you please kindly explain what a Python decorator is?
After  (6 words):  Explain what a python decorator is?
```

Real World: *This Map-Reduce pattern is used by NotebookLM, ChatGPT's document analysis, and every AI that processes PDFs. LangChain's RecursiveCharacterTextSplitter does exactly this chunking under the hood.*

Mini Challenge:

1. Read a real .txt file from disk using open('file.txt').read() and run it through summarize_long_document()
2. Change chunk_size to 50 words and 500 words. Compare: quality vs cost tradeoff
3. Add a progress percentage: 'Processing chunk 2/5 (40%)...'

Section 8 (4.3): Cost Estimation & Budget Control

Track every rupee spent on AI API calls

Why You MUST Track Costs in Production

The Horror Story (it happens to everyone):

Developer launches an AI app. Gets popular. Wakes up to a $3,000 AWS bill because 10,000 users each triggered 5,000-token prompts overnight. No budget cap. No tracking.

This happens EVERY MONTH to someone. Don't be that person.

The Solution: Track Before + After Every Call

1. Count tokens BEFORE calling (predict cost)
2. Record tokens AFTER calling (actual cost)
3. Maintain running total per user/session
4. Set hard budget limits that REJECT calls when exceeded

Python Concepts Used: `class, dataclass, @dataclass, __post_init__, dict, list, sum(), property, exception`

```python
# ── cost_tracker.py ──────────────────────────────────────────
# Production-grade cost tracker with budget enforcement

import google.generativeai as genai
import os, time, json
from dataclasses import dataclass, field  # dataclass = class with less boilerplate
from dotenv import load_dotenv
```

```python
load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

# ── Pricing constants (Gemini 1.5 Flash, June 2025) ──────────────────
PRICE_INPUT_PER_TOKEN  = 0.075 / 1_000_000   # $0.075 per million
PRICE_OUTPUT_PER_TOKEN = 0.300 / 1_000_000   # $0.30 per million
USD_TO_INR             = 83.5                 # approximate rate

# ── @dataclass auto-generates __init__, __repr__, __eq__ ─────────────
@dataclass
class APICall:
    '''Stores data for a single API call. @dataclass saves writing __init__.'''
    prompt:          str
    input_tokens:    int
    output_tokens:   int
    cost_usd:        float
    timestamp:       float = field(default_factory=time.time)  # auto-set
    response_text:   str   = ''

    @property
    def cost_inr(self) -> float:
        '''Convert USD cost to INR automatically.'''
        return self.cost_usd * USD_TO_INR

    @property
    def total_tokens(self) -> int:
        return self.input_tokens + self.output_tokens

class BudgetAwareClient:
    '''
    A wrapper around Gemini that tracks every API call.
    Raises BudgetExceededError when spending limit is hit.
    '''

    class BudgetExceededError(Exception):
        '''Custom exception for budget violations.'''
        pass

    def __init__(self, budget_usd: float = 1.0):
        self.model      = genai.GenerativeModel('gemini-1.5-flash')
        self.budget_usd = budget_usd           # hard cap
        self.calls: list[APICall] = []         # history of all calls

    @property
    def total_spent_usd(self) -> float:
        '''Sum of all call costs — using built-in sum() with generator.'''
        return sum(call.cost_usd for call in self.calls)  # generator expr

    @property
    def budget_remaining_usd(self) -> float:
        return self.budget_usd - self.total_spent_usd

    @property
    def call_count(self) -> int:
        return len(self.calls)

    def ask(self, prompt: str, max_tokens: int = 300) -> str:
        '''
        Call Gemini with full cost tracking and budget enforcement.
        Raises BudgetExceededError if over budget BEFORE calling.
        '''
```

```
        # 1. Predict cost before calling
        token_count  = self.model.count_tokens(prompt).total_tokens
        est_out      = min(max_tokens, 300)   # estimate output
        est_cost     = (token_count * PRICE_INPUT_PER_TOKEN +
                        est_out   * PRICE_OUTPUT_PER_TOKEN)

        # 2. Check budget BEFORE making expensive API call
        if self.total_spent_usd + est_cost > self.budget_usd:
            raise self.BudgetExceededError(
                f'Budget exceeded! Spent: ${self.total_spent_usd:.6f} / '
                f'${self.budget_usd:.3f}. Estimated next call: ${est_cost:.6f}'
            )

        # 3. Make the actual API call
        config   = genai.GenerationConfig(max_output_tokens=max_tokens)
        response = self.model.generate_content(prompt, generation_config=config)

        # 4. Record actual cost (not estimate)
        actual_in  = response.usage_metadata.prompt_token_count
        actual_out = response.usage_metadata.candidates_token_count
        actual_cost = (actual_in  * PRICE_INPUT_PER_TOKEN +
                       actual_out * PRICE_OUTPUT_PER_TOKEN)

        # 5. Store the call record
        call = APICall(
            prompt=prompt[:50],       # truncate for storage
            input_tokens=actual_in,
            output_tokens=actual_out,
            cost_usd=actual_cost,
            response_text=response.text
        )
        self.calls.append(call)
        return response.text

    def print_report(self):
        '''Print a formatted spending report.'''
        print('\n' + '='*55)
        print(f'  COST REPORT  |  Budget: ${self.budget_usd:.3f}')
        print('='*55)
        print(f'  {'Call':<4}  {'Tokens In':>9}  {'Tokens Out':>10}  {'Cost USD':>10}
{'Cost INR':>9}')
        print('-'*55)
        for i, c in enumerate(self.calls, 1):   # enumerate for numbering
            print(f'  {i:<4}  {c.input_tokens:>9}  {c.output_tokens:>10}  '
                  f'${c.cost_usd:>9.6f}  ₹{c.cost_inr:>8.4f}')
        print('-'*55)
        print(f'  {'TOTAL':<4}  {'':>9}  {'':>10}  '
              f'${self.total_spent_usd:>9.6f}  ₹{self.total_spent_usd *
USD_TO_INR:>8.4f}')
        print(f'  Remaining: ${self.budget_remaining_usd:.6f}')
        print('='*55)

# ── Demo ──────────────────────────────────────────────────────────────
if __name__ == '__main__':
    client = BudgetAwareClient(budget_usd=0.001)  # 0.1 cent budget for demo

    questions = [
        'What is Python?',
        'What is a list comprehension?',
        'What is recursion?',
        'Explain the entire history of computing in 2000 words.',  # this will exceed
    ]
```

```python
    for q in questions:
        try:
            answer = client.ask(q, max_tokens=80)
            print(f'Q: {q[:40]}')
            print(f'A: {answer[:80]}...')
            print(f'   [Spent so far: ${client.total_spent_usd:.6f}]\n')
        except BudgetAwareClient.BudgetExceededError as e:
            print(f'BLOCKED: {e}')
            break

    client.print_report()
```

Output:

```
Q: What is Python?
A: Python is a high-level, interpreted programming language known for...
   [Spent so far: $0.000024]

Q: What is a list comprehension?
A: A list comprehension is a concise way to create lists in Python using...
   [Spent so far: $0.000051]

Q: What is recursion?
A: Recursion is when a function calls itself to solve a smaller version...
   [Spent so far: $0.000078]

BLOCKED: Budget exceeded! Spent: $0.000078 / $0.001. Estimated: $0.000248

============================================================
  COST REPORT  |  Budget: $0.001
============================================================
  Call  Tokens In  Tokens Out    Cost USD   Cost INR
------------------------------------------------------------
  1             5          62  $0.000024  ₹0.0020
  2             7          71  $0.000027  ₹0.0023
  3             5          69  $0.000027  ₹0.0023
------------------------------------------------------------
  TOTAL                        $0.000078  ₹0.0065
  Remaining: $0.000922
============================================================
```

Real World: *Every serious AI company uses budget tracking. OpenAI charges you by the token — no cap by default. Startups have gone bankrupt from runaway API costs. This exact pattern (check before call, record after) is used in production systems at Google, Anthropic, and OpenAI's enterprise customers.*

Mini Challenge:

1. Add a per_call_limit_usd parameter that blocks any single call costing more than X dollars
2. Save the spending report to a CSV file using Python's csv module
3. Add a warning at 80% budget usage instead of only blocking at 100%

Section 9 (4.4): Prompt Caching — Pay Once, Use Forever

The biggest cost saving trick in production AI systems

What is Prompt Caching?

Without caching:

`1000 users all send: 'You are a helpful Arya College AI tutor...[500 token system message]...What is Python?'`

= 1000 × 500 input tokens = 500,000 tokens JUST for the system message.

With caching:

System message is processed ONCE and cached. Next 999 users get it from cache at 75-90% lower cost.

= 1 × 500 tokens normal + 999 × 500 tokens at 10% price. Savings: ~90%

Python-Level Caching: Using hashlib + dict

Even without official API-level caching, we can cache at the Python level using a hash map. Same prompt = same response (for deterministic tasks like classifications).

Python Concepts Used: `hashlib.md5, dict, functools.lru_cache, time.time(), dataclass, json.dumps`

```
# ── prompt_cache.py ──────────────────────────────────────────
# Multi-layer caching: Python dict + LRU + TTL (time-to-live)

import google.generativeai as genai
import os, time, hashlib, json
from functools import lru_cache
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
model = genai.GenerativeModel('gemini-1.5-flash')

# ── Cache Entry: stores response + when it was cached ────────
@dataclass
class CacheEntry:
    response: str
    cached_at: float      # timestamp when cached
    hit_count: int = 0    # how many times this was served from cache

    def is_fresh(self, ttl_seconds: int = 3600) -> bool:
        '''Check if cache entry is still valid (not expired).'''
        age = time.time() - self.cached_at          # age in seconds
        return age < ttl_seconds                     # True if fresh

class SmartCache:
    '''
    Caches AI responses by prompt hash.
    Hash = fingerprint of the text. Same text → same hash → cache hit.
    '''
```

```
    def __init__(self, ttl_seconds: int = 3600, max_size: int = 500):
        self._cache: dict[str, CacheEntry] = {}     # hash → CacheEntry
        self.ttl         = ttl_seconds
        self.max_size    = max_size
        self.total_hits  = 0
        self.total_misses = 0
        self.tokens_saved = 0

    def _hash(self, text: str) -> str:
        '''
        Create a unique fingerprint of any text string.
        hashlib.md5 → 128-bit hash → 32 hex characters.
        Same text always → same hash. Different text → different hash.
        '''
        return hashlib.md5(text.encode('utf-8')).hexdigest()

    def get(self, prompt: str) -> str | None:
        '''Look up prompt in cache. Returns response or None.'''
        key   = self._hash(prompt)
        entry = self._cache.get(key)              # dict.get() returns None if missing

        if entry is None:
            self.total_misses += 1
            return None

        if not entry.is_fresh(self.ttl):
            del self._cache[key]                  # expired: delete from dict
            self.total_misses += 1
            return None

        # Cache hit!
        entry.hit_count  += 1
        self.total_hits  += 1
        self.tokens_saved += len(prompt.split()) // 3 * 4  # rough estimate
        return entry.response

    def set(self, prompt: str, response: str) -> None:
        '''Store a response in cache.'''
        # Evict oldest entries if at max size
        if len(self._cache) >= self.max_size:
            oldest_key = min(
                self._cache.keys(),
                key=lambda k: self._cache[k].cached_at   # lambda for sorting
            )
            del self._cache[oldest_key]

        key              = self._hash(prompt)
        self._cache[key] = CacheEntry(response=response, cached_at=time.time())

    @property
    def hit_rate(self) -> float:
        '''What % of requests were served from cache (free).'''
        total = self.total_hits + self.total_misses
        return (self.total_hits / total * 100) if total > 0 else 0.0

    def stats(self) -> dict:
        return {
            'hits':        self.total_hits,
            'misses':      self.total_misses,
            'hit_rate':    f'{self.hit_rate:.1f}%',
            'cache_size':  len(self._cache),
            'tokens_saved': self.tokens_saved,
```

```
        }

# ── Cached AI Client ──────────────────────────────────────────
cache = SmartCache(ttl_seconds=3600, max_size=200)

def ask_with_cache(prompt: str, temperature: float = 0.3) -> dict:
    '''
    Ask AI but check cache first.
    Low temperature (0.3) = deterministic = cacheable.
    High temperature (1.5) = random = NOT worth caching.
    '''
    # Check cache FIRST
    cached = cache.get(prompt)
    if cached:
        return {'text': cached, 'source': 'CACHE', 'tokens': 0}

    # Cache miss → call API
    config   = genai.GenerationConfig(temperature=temperature, max_output_tokens=150)
    response = model.generate_content(prompt, generation_config=config)
    text     = response.text.strip()
    tokens   = response.usage_metadata.prompt_token_count

    # Store in cache for next time
    cache.set(prompt, text)

    return {'text': text, 'source': 'API', 'tokens': tokens}

# ── Demo: Simulate 10 users asking the same 3 questions ──────
if __name__ == '__main__':
    faq_questions = [
        'What is Python?',
        'What is a variable in programming?',
        'What is a function in Python?',
    ]

    print('Simulating 10 users asking FAQ questions...')
    print('(First time each question = API call. After = cache hit)\n')

    import random
    for user_num in range(1, 11):   # 10 simulated users
        q = random.choice(faq_questions)    # random question
        result = ask_with_cache(q)
        src    = result['source']
        icon   = '□ API' if src == 'API' else '⚡ CACHE'
        print(f'  User {user_num:02d}: {icon} | Q: {q[:35]:<35} | Tokens:
{result["tokens"]}')

    # Print cache stats
    stats = cache.stats()
    print(f'\n{'='*50}')
    print(f'Cache Stats: {json.dumps(stats, indent=2)}')
```

□ Output:

```
Simulating 10 users asking FAQ questions...
(First time each question = API call. After = cache hit)

  User 01: □ API   | Q: What is Python?                  | Tokens: 4
  User 02: ⚡ CACHE  | Q: What is Python?                  | Tokens: 0
  User 03: □ API   | Q: What is a variable in programmin | Tokens: 8
  User 04: ⚡ CACHE  | Q: What is Python?                  | Tokens: 0
```

```
  User 05: □ API   | Q: What is a function in Python?    | Tokens: 7
  User 06: ⚡ CACHE  | Q: What is a function in Python?   | Tokens: 0
  User 07: ⚡ CACHE  | Q: What is a variable in programmin| Tokens: 0
  User 08: ⚡ CACHE  | Q: What is Python?                 | Tokens: 0
  User 09: ⚡ CACHE  | Q: What is a function in Python?   | Tokens: 0
  User 10: ⚡ CACHE  | Q: What is a variable in programmin| Tokens: 0

==================================================
Cache Stats: {
  "hits": 7,
  "misses": 3,
  "hit_rate": "70.0%",
  "cache_size": 3,
  "tokens_saved": 140
}
```

□ **Real World:** *Perplexity AI, You.com, and every AI FAQ system uses this pattern. When 10,000 students all ask 'What is Python?', you only pay for it ONCE. Redis is used in production instead of a Python dict, but the logic is identical.*

□ **Mini Challenge:**

1. Add a cache_to_file() method that saves the cache to a JSON file so it survives program restarts
2. Add a 'never_cache' list for questions with temperature > 1.0 (creative answers shouldn't be cached)
3. Make the TTL configurable per-question: technical FAQs cache for 7 days, news summaries for 1 hour

Section 10 (4.5): Rate Limiting & Retry Logic

Handle API failures gracefully — the mark of a professional

Two Problems Every Production AI App Faces:

Problem 1: Rate Limit Errors (429)

Gemini Free tier: 15 requests/minute. If 50 users ask at once, 35 get a 429 error. Without retry logic, they see a crash. With it, they wait briefly and succeed.

Problem 2: Network Timeouts (503)

Internet drops for 2 seconds. API call fails. Without retry → user sees error. With exponential backoff → auto-retries at 1s, 2s, 4s, 8s gaps. Success.

□ **Python Concepts Used:** `time.sleep()`, `try/except/else`, `for loop`, `min()`, `pow()`, `random.uniform`, `functools.wraps`, `decorator`

```python
# ── rate_limiter.py ──────────────────────────────────────────
# Retry with exponential backoff + rate limiter in one class

import google.generativeai as genai
import os, time, random
from functools import wraps
from dotenv import load_dotenv
```

```
load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
model = genai.GenerativeModel('gemini-1.5-flash')

# ── Decorator: automatically retries any function ──────────────────────
def with_retry(max_attempts: int = 4, base_delay: float = 1.0):
    '''
    Decorator factory: returns a decorator that adds retry logic.
    Uses exponential backoff: waits 1s, 2s, 4s, 8s between retries.
    @wraps preserves the original function's name and docstring.
    '''
    def decorator(func):
        @wraps(func)           # keeps func.__name__ and func.__doc__
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)    # try calling the function
                except Exception as e:
                    last_error = e
                    error_msg  = str(e).lower()

                    # Check if it's worth retrying
                    if '429' in error_msg or 'quota' in error_msg:
                        delay = base_delay * pow(2, attempt - 1)  # 1,2,4,8...
                        jitter = random.uniform(0, 0.5)            # randomness
                        wait   = min(delay + jitter, 30)           # cap at 30s
                        print(f'    [Retry {attempt}/{max_attempts}] Rate limited.
Waiting {wait:.1f}s...')
                        time.sleep(wait)
                    elif '503' in error_msg or 'timeout' in error_msg:
                        wait = base_delay * attempt    # linear for timeouts
                        print(f'    [Retry {attempt}/{max_attempts}] Service error.
Waiting {wait:.1f}s...')
                        time.sleep(wait)
                    else:
                        raise   # not retriable — raise immediately

            raise RuntimeError(f'Failed after {max_attempts} attempts: {last_error}')
        return wrapper
    return decorator

# ── Rate Limiter: controls requests-per-minute ─────────────────────────
class RateLimiter:
    '''
    Token bucket algorithm: allows max_calls per minute.
    Tracks timestamps of recent calls. Waits if too many recent calls.
    '''

    def __init__(self, max_calls_per_minute: int = 12):  # 12 = safe below 15 limit
        self.max_calls  = max_calls_per_minute
        self.window     = 60.0                    # 1 minute window
        self.call_times: list[float] = []         # timestamps of recent calls

    def wait_if_needed(self):
        '''Block execution if we're making calls too fast.'''
        now = time.time()

        # Remove calls older than 1 minute from our tracking list
        self.call_times = [t for t in self.call_times if now - t < self.window]

        if len(self.call_times) >= self.max_calls:
```

```
            # Too many recent calls — wait until oldest one expires
            oldest  = self.call_times[0]          # first (oldest) timestamp
            wait    = self.window - (now - oldest) + 0.1
            if wait > 0:
                print(f'    [Rate Limiter] At {self.max_calls}/min limit. Waiting
{wait:.1f}s...')
                time.sleep(wait)

        self.call_times.append(time.time())

# ── Combined: Retry + Rate Limiting ──────────────────────────────
limiter = RateLimiter(max_calls_per_minute=12)

@with_retry(max_attempts=4, base_delay=1.0)    # retry up to 4 times
def safe_ask(prompt: str, max_tokens: int = 150) -> str:
    '''Rate-limited + auto-retrying AI call.'''
    limiter.wait_if_needed()          # check rate limit FIRST
    config   = genai.GenerationConfig(max_output_tokens=max_tokens, temperature=0.5)
    response = model.generate_content(prompt, generation_config=config)
    return response.text.strip()

# ── Demo: Make 5 rapid calls that would fail without rate limiter ────────
if __name__ == '__main__':
    questions = [
        'What is Python?',
        'What is a list?',
        'What is a dict?',
        'What is a function?',
        'What is a class?',
    ]

    print('Making 5 API calls with rate limiting + auto-retry:')
    for i, q in enumerate(questions, 1):
        start  = time.time()
        answer = safe_ask(q)
        elapsed = time.time() - start
        print(f'  [{i}] {q:<30} → {elapsed:.2f}s → {answer[:50]}...')
```

Output:

```
Making 5 API calls with rate limiting + auto-retry:
  [1] What is Python?              → 0.31s → Python is a high-level, interpreted...
  [2] What is a list?              → 0.28s → A list is an ordered, mutable collection...
  [3] What is a dict?              → 0.29s → A dictionary (dict) is an unordered
collection...
  [4] What is a function?          → 0.31s → A function is a reusable block of code...
  [5] What is a class?             → 0.33s → A class is a blueprint for creating
objects...
```

Real World: *This @with_retry decorator pattern is used in every production API client: Stripe's SDK, Twilio's Python client, AWS Boto3. The exponential backoff (1s, 2s, 4s, 8s) is a distributed systems standard — it prevents thundering herd problems when everyone retries at the same time.*

Mini Challenge:

1. Add a circuit breaker: after 5 consecutive failures, stop all calls for 60 seconds
2. Add logging: write every retry attempt to a file called retries.log with timestamp
3. Apply @with_retry to your BudgetAwareClient.ask() from Section 8 — combine both features

Section 11: Structured Output — Getting JSON from AI

Make AI return data you can actually use in your app

Why Structured Output?

Problem with Free-Text AI Output:

'Summarize this article' → returns a paragraph. Great for reading. Useless for your database.

`You can't do: summary.title or summary['sentiment'] on plain text.`

Solution: Force AI to respond in JSON:

Tell the AI: 'Respond ONLY with valid JSON, no other text.'

`Now you can: data['title'], data['sentiment'], save to MongoDB.`

Python Concepts Used: `json.loads()`, `json.dumps()`, `try/except`, `re.search()`, `@dataclass`, `TypedDict`

```python
# ── structured_output.py ──────────────────────────────────────
# Force AI to return JSON data you can use in your application

import google.generativeai as genai
import os, json, re
from dataclasses import dataclass
from typing import TypedDict
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

# ── Define the structure we want AI to return ─────────────────
class ArticleSummary(TypedDict):   # TypedDict: dict with defined key types
    title:      str
    summary:    str
    sentiment:  str    # 'positive' | 'negative' | 'neutral'
    keywords:   list   # list of strings
    difficulty: int    # 1-5 reading level

# ── Generic JSON extractor ────────────────────────────────────
def extract_json(text: str) -> dict | None:
    '''
    AI sometimes wraps JSON in ```json ... ``` — strip that first.
    Then parse with json.loads(). Returns None if parsing fails.
    '''
    # Remove ```json ... ``` wrappers if present
    text = re.sub(r'```json\s*', '', text)    # re.sub: regex replace
    text = re.sub(r'```\s*',     '', text)
    text = text.strip()

    try:
        return json.loads(text)      # parse JSON string → Python dict
    except json.JSONDecodeError:
        # Try finding JSON inside the text (AI may add explanation)
        match = re.search(r'\{.*\}', text, re.DOTALL)   # DOTALL: match newlines
```

```
        if match:
            try:
                return json.loads(match.group())
            except:
                return None
    return None

def summarize_article(article_text: str) -> ArticleSummary | None:
    '''
    Send article to AI, get back structured data as Python dict.
    The trick: very explicit JSON schema in the prompt.
    '''
    model = genai.GenerativeModel(
        model_name='gemini-1.5-flash',
        system_instruction='''You are a JSON-only response bot.
        ALWAYS respond with ONLY valid JSON. No other text.
        No explanations. No markdown. ONLY the JSON object.'''
    )

    prompt = f'''Analyze this article and respond with ONLY this JSON structure:
    {{
      "title":      "string — a 5-word title you invent",
      "summary":    "string — 2 sentence summary",
      "sentiment":  "positive" or "negative" or "neutral",
      "keywords":   ["word1", "word2", "word3"],
      "difficulty": number from 1 (easy) to 5 (expert)
    }}

    ARTICLE:
    {article_text[:1000]}

    Respond with ONLY the JSON. No other text:'''

    config = genai.GenerationConfig(temperature=0.2, max_output_tokens=300)
    response = model.generate_content(prompt, generation_config=config)
    return extract_json(response.text)

def classify_question(question: str) -> dict | None:
    '''
    Classify a student question into category + difficulty + suggested topics.
    Useful for a smart tutoring system that routes questions.
    '''
    model = genai.GenerativeModel('gemini-1.5-flash')
    prompt = f'''Classify this student question. ONLY respond with JSON:
    {{
      "category":    "python_basics" | "oop" | "algorithms" | "web" | "ai" | "other",
      "difficulty":  "beginner" | "intermediate" | "advanced",
      "topics":      ["topic1", "topic2"],
      "can_answer":  true | false
    }}

    Question: {question}
    JSON only:'''

    config   = genai.GenerationConfig(temperature=0.1, max_output_tokens=150)
    response = model.generate_content(prompt, generation_config=config)
    return extract_json(response.text)

# ── Demo ──────────────────────────────────────────────────────────
if __name__ == '__main__':
    # Demo 1: Article summarization
    article = '''
```

```
    Python 3.12 was released with major performance improvements.
    The new version is 60% faster for most workloads due to a new
    interpreter design. Key features include improved error messages,
    f-string improvements, and a new type system. Developers are
    excited about the speed gains for web applications.
    '''

    print('=== STRUCTURED ARTICLE SUMMARY ===')
    summary = summarize_article(article)
    if summary:
        print(f'Title:      {summary["title"]}')
        print(f'Sentiment:  {summary["sentiment"]}')
        print(f'Keywords:   {", ".join(summary["keywords"])}')
        print(f'Difficulty: {summary["difficulty"]}/5')
        print(f'Summary:    {summary["summary"]}')

    # Demo 2: Question classification
    print('\n=== QUESTION CLASSIFICATION ===')
    questions = [
        'What is a Python list comprehension?',
        'How do I implement a binary search tree?',
        'What is gradient descent?',
    ]
    for q in questions:
        result = classify_question(q)
        if result:
            print(f'Q: {q}')
            print(f'   Category: {result["category"]} | Difficulty:
{result["difficulty"]}')
            print(f'   Topics: {", ".join(result["topics"])}')
```

Output:

```
=== STRUCTURED ARTICLE SUMMARY ===
Title:      Python 3.12 Achieves Record Speed
Sentiment:  positive
Keywords:   Python, performance, interpreter, speed, web
Difficulty: 2/5
Summary:    Python 3.12 brings 60% speed improvements via a redesigned interpreter.
            Developers benefit from better error messages and f-string enhancements.

=== QUESTION CLASSIFICATION ===
Q: What is a Python list comprehension?
   Category: python_basics | Difficulty: beginner
   Topics: list, comprehension, iteration
Q: How do I implement a binary search tree?
   Category: algorithms | Difficulty: advanced
   Topics: binary tree, data structures, recursion
Q: What is gradient descent?
   Category: ai | Difficulty: intermediate
   Topics: optimization, machine learning, mathematics
```

Real World: *EVERY modern AI product uses structured output. When ChatGPT fills a form, Gemini extracts data from an invoice, or GitHub Copilot generates function signatures — they're all using this JSON mode pattern. Zomato's AI menu parser returns {dish_name, price, calories, allergens} for every food photo.*

Section 12: CAPSTONE — AI Study Assistant Chatbot

Combines EVERYTHING from Lectures 15 & 16 into one production system

Project: AryaBot — An Intelligent Study Assistant

What AryaBot does:

1. Streaming responses (users see answers word-by-word like ChatGPT)
2. Multi-turn conversation memory (remembers what you asked before)
3. Smart caching (same FAQ = free response from cache)
4. Budget control (never spends more than configured limit)
5. Rate limiting + auto-retry (handles 429s gracefully)
6. Question classification (routes easy vs hard questions differently)
7. Structured logging (JSON log of every session for analytics)

Component	From Section	Python Concept	Production Use
`StreamingChatbot`	Section 5	`class, yield, history list`	*ChatGPT architecture*
`SmartCache`	Section 9	`hashlib, dict, dataclass`	*Redis in production*
`BudgetAwareClient`	Section 8	`@property, exception`	*OpenAI billing guards*
`RateLimiter`	Section 10	`list, time.time()`	*Nginx rate limiting*
`classify_question`	Section 11	`json, regex, TypedDict`	*Routing systems*
`with_retry decorator`	Section 10	`@wraps, closures`	*AWS SDK pattern*
`chunk_text`	Section 7	`split(), range(), zip()`	*LangChain splitter*

Complete AryaBot Implementation:

```
# ── arya_bot.py ──────────────────────────────────────────────────
# CAPSTONE: Full AI Study Assistant — combines everything from Lectures 15 & 16

import google.generativeai as genai
import os, time, json, hashlib, re, random, math
from dataclasses import dataclass, field
from functools import wraps
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
```

```python
# ============================================================
# COMPONENT 1: SMART CACHE (Section 9)
# ============================================================
@dataclass
class CacheEntry:
    response: str
    cached_at: float = field(default_factory=time.time)
    hit_count: int   = 0

    def is_fresh(self, ttl: int = 1800) -> bool:
        return (time.time() - self.cached_at) < ttl

class SmartCache:
    def __init__(self): self._store: dict = {}; self.hits = 0; self.misses = 0

    def _key(self, text: str) -> str:
        return hashlib.md5(text.strip().lower().encode()).hexdigest()

    def get(self, text: str) -> str | None:
        entry = self._store.get(self._key(text))
        if entry and entry.is_fresh():
            entry.hit_count += 1; self.hits += 1; return entry.response
        self.misses += 1; return None

    def set(self, text: str, response: str):
        self._store[self._key(text)] = CacheEntry(response=response)

    @property
    def hit_rate(self) -> str:
        total = self.hits + self.misses
        return f'{self.hits/total*100:.0f}%' if total else '0%'

# ============================================================
# COMPONENT 2: RETRY DECORATOR (Section 10)
# ============================================================
def with_retry(max_attempts=3, base_delay=1.0):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts: raise
                    wait = base_delay * pow(2, attempt - 1)
                    if '429' in str(e): time.sleep(wait)
                    else: raise
        return wrapper
    return decorator

# ============================================================
# COMPONENT 3: QUESTION CLASSIFIER (Section 11)
# ============================================================
def classify_question(question: str, model) -> dict:
    '''Route questions: simple → short answer, complex → detailed.'''
    prompt = f'''Classify this CS student question. JSON only:
    {{"difficulty":"easy"|"medium"|"hard","needs_code":true|false,"topic":"string"}}
    Question: {question}'''
    cfg  = genai.GenerationConfig(temperature=0.1, max_output_tokens=80)
    resp = model.generate_content(prompt, generation_config=cfg)
    try:
```

```
        text = re.sub(r'```json|```','', resp.text).strip()
        return json.loads(text)
    except:
        return {'difficulty': 'medium', 'needs_code': False, 'topic': 'general'}

# ═══════════════════════════════════════════════════════
# MAIN CLASS: ArYABOT
# ═══════════════════════════════════════════════════════
class AryaBot:
    '''
    Full AI Study Assistant for Arya College of Engineering.
    Combines: streaming + caching + budget + retry + classification.
    '''

    # Gemini 1.5 Flash pricing
    PRICE_IN  = 0.075 / 1_000_000
    PRICE_OUT = 0.300 / 1_000_000

    SYSTEM_MESSAGE = '''You are ArYABot, the AI study assistant for Arya College
    of Engineering's Agentic AI Industrial Training course.
    You help 2nd-3rd year B.Tech students learn Python and AI.
    Rules:
    - Use Hinglish naturally (mix Hindi + English)
    - Always include a small Python code example if relevant
    - Keep answers under 6 sentences unless asked for more
    - End each answer with a follow-up question to check understanding
    - If you don't know something, say 'Yaar, iska mujhe confirm nahi' honestly
    '''

    def __init__(self, budget_usd: float = 0.50):
        # The main model with system message
        self.model = genai.GenerativeModel(
            model_name='gemini-1.5-flash',
            system_instruction=self.SYSTEM_MESSAGE
        )
        # The classifier model (no system message needed)
        self.classifier = genai.GenerativeModel('gemini-1.5-flash')

        # State
        self.cache    = SmartCache()
        self.budget   = budget_usd
        self.spent    = 0.0
        self.history  : list[dict] = []
        self.session_log: list[dict] = []
        self.turn     = 0

    @property
    def budget_ok(self) -> bool:
        return self.spent < self.budget

    @property
    def stats(self) -> dict:
        return {
            'turns':       self.turn,
            'spent_usd':   round(self.spent, 6),
            'budget_usd':  self.budget,
            'cache_hits':  self.cache.hits,
            'cache_rate':  self.cache.hit_rate,
        }

    def _build_prompt(self, user_msg: str) -> str:
        '''Build full prompt with conversation history.'''
```

```
        lines = []
        # Only keep last 6 turns to save tokens (sliding window)
        recent = self.history[-6:]       # slice: last 6 items
        for msg in recent:
            role = 'User' if msg['role'] == 'user' else 'ArYABot'
            lines.append(f'{role}: {msg["content"]}')
        lines.append(f'User: {user_msg}')
        lines.append('ArYABot:')
        return '\n'.join(lines)

    @with_retry(max_attempts=3, base_delay=1.0)
    def _api_call_stream(self, prompt: str, config):
        '''Wrapped in retry decorator automatically.'''
        return self.model.generate_content(prompt, generation_config=config,
stream=True)

    def chat(self, user_message: str):
        '''
        Main entry point. Returns a generator that yields response chunks.
        1. Check cache (free!)
        2. Classify question (set appropriate temperature + max_tokens)
        3. Build context-aware prompt
        4. Stream response
        5. Track cost
        6. Save to history
        '''
        if not self.budget_ok:
            yield '[Budget exhausted. Please contact your instructor.]'
            return

        self.turn += 1
        log_entry = {'turn': self.turn, 'user': user_message, 'source': None}

        # STEP 1: Check cache
        cached = self.cache.get(user_message)
        if cached:
            log_entry['source'] = 'cache'
            self.session_log.append(log_entry)
            yield from cached.split(' ')  # stream cached response word-by-word
            return

        # STEP 2: Classify question → adaptive parameters
        meta = classify_question(user_message, self.classifier)
        difficulty   = meta.get('difficulty', 'medium')
        needs_code   = meta.get('needs_code', False)

        # Adaptive generation config based on question type
        if difficulty == 'easy':
            temperature = 0.4; max_tokens = 150
        elif difficulty == 'hard':
            temperature = 0.6; max_tokens = 500
        else:
            temperature = 0.5; max_tokens = 300

        if needs_code: max_tokens = int(max_tokens * 1.5)   # more tokens for code

        # STEP 3: Build prompt
        full_prompt = self._build_prompt(user_message)
        config = genai.GenerationConfig(
            temperature=temperature,
            max_output_tokens=max_tokens
        )
```

```
        # STEP 4: Stream response
        response     = self._api_call_stream(full_prompt, config)
        full_response = ''

        for chunk in response:
            if chunk.text:
                full_response += chunk.text
                yield chunk.text    # stream to caller

        # STEP 5: Track cost
        try:
            in_tok  = response.usage_metadata.prompt_token_count
            out_tok = response.usage_metadata.candidates_token_count
            cost    = in_tok * self.PRICE_IN + out_tok * self.PRICE_OUT
            self.spent += cost
            log_entry['cost_usd'] = round(cost, 7)
            log_entry['tokens']   = {'in': in_tok, 'out': out_tok}
        except:
            pass

        # STEP 6: Update history + cache
        self.history.append({'role': 'user',      'content': user_message})
        self.history.append({'role': 'assistant', 'content': full_response})
        self.cache.set(user_message, full_response)   # cache for next user
        log_entry['source'] = 'api'
        self.session_log.append(log_entry)

    def reset(self):
        '''Clear conversation but keep cache (shared across sessions).'''
        self.history = []
        self.turn = 0
        print('[ArYABot] Memory cleared. Cache preserved for speed.')

    def save_log(self, filename: str = 'session_log.json'):
        '''Save session analytics to JSON file.'''
        with open(filename, 'w') as f:
            json.dump({'stats': self.stats, 'log': self.session_log}, f, indent=2)
        print(f'[Log saved to {filename}]')


# ═══════════════════════════════════════════════════════════
# RUN ARYA BOT
# ═══════════════════════════════════════════════════════════
if __name__ == '__main__':
    bot = AryaBot(budget_usd=0.50)

    print('╔═══════════════════════════════════════════════════╗')
    print('║       ArYABot — Arya College AI Study Assistant       ║')
    print('║  Commands: reset | stats | save | quit                ║')
    print('╚═══════════════════════════════════════════════════╝')
    print()

    while True:
        try:
            user_input = input('You: ').strip()
        except (EOFError, KeyboardInterrupt):
            print('\nGoodbye!')
            break

        if not user_input: continue
```

```
        if user_input.lower() == 'quit':
            bot.save_log()
            print(f'Session complete! Stats: {bot.stats}')
            break
        elif user_input.lower() == 'reset':
            bot.reset(); continue
        elif user_input.lower() == 'stats':
            print(f'\n Stats: {json.dumps(bot.stats, indent=2)}\n'); continue
        elif user_input.lower() == 'save':
            bot.save_log(); continue

        # Stream the response
        print('\nArYABot: ', end='', flush=True)
        for chunk in bot.chat(user_input):
            print(chunk, end='', flush=True)
        print('\n')
```

Output:

```
╔═══════════════════════════════════════════════════╗
║       ArYABot — Arya College AI Study Assistant       ║
║  Commands: reset | stats | save | quit                ║
╚═══════════════════════════════════════════════════╝


You: decorators kya hote hain Python mein?

ArYABot: Decorator ek aisa function hai jo doosre function ko 'wrap' karta
hai — jaise gift wrap karte hain present ko!  Isko @symbol se likhte hain.
Real life mein: @login_required decorator check karta hai ki user logged in
hai ya nahi, BEFORE the actual view function runs.
```python
def my_decorator(func):
    def wrapper(): print('Before'); func(); print('After')
    return wrapper
@my_decorator
def hello(): print('Hello!')
hello()  # prints: Before, Hello!, After
```
Tumne kabhi @staticmethod ya @property use kiya hai? Woh bhi decorators hain!

You: stats

 Stats: {
  "turns": 1,
  "spent_usd": 0.000089,
  "budget_usd": 0.5,
  "cache_hits": 0,
  "cache_rate": "0%"
}
```

Real World: *ArYABot's architecture is identical to production chatbots: Anthropic's Claude, ChatGPT, Gemini all use this exact pattern — streaming generator + conversation history list + system message + cost tracking. The only difference is scale: they handle millions of simultaneous users.*

Section 13: Python Concepts Quick Reference

Every Python feature used in this guide — in one place

Part 1: Core Python Features

Concept	Syntax	Used in this guide for
f-string	`f'hello {name}'`	Building prompts with variables
list comprehension	`[x*2 for x in lst]`	Filtering chunks, mapping results
generator / yield	`def f(): yield x`	Streaming AI responses chunk-by-chunk
@decorator	`@with_retry(3)`	Adding retry/logging to any function
@dataclass	`@dataclass class X`	APICall, CacheEntry — less boilerplate
@property	`@property def x`	budget_remaining, hit_rate — computed attrs
@lru_cache	`@lru_cache(256)`	Token counting — instant repeated calls
TypedDict	`class X(TypedDict)`	Structured AI output schemas
try/except	`try: ... except E:`	Handling 429, JSON parse errors
***args/**kwargs**	`def f(*a, **k)`	Retry wrapper accepts any function
zip()	`zip(temps, labels)`	Pair temperature values with labels
enumerate()	`for i,x in enumerate(lst)`	Numbered chunk processing
lambda	`lambda k: d[k].ts`	Inline sorting key for cache eviction
hashlib.md5	`hashlib.md5(t.encode())`	Create cache keys from text
json.loads/dumps	`json.loads(text)`	Parse AI JSON output to Python dict
time.time()	`time.time()`	Timestamps, TTL checks, rate limiting
re.sub/search	`re.sub(pattern,'',t)`	Strip ```json wrappers from AI output
random.choice	`random.choice(lst)`	Select questions in demos

Part 2: Gemini API Reference

API Feature	Code Example
Create model	`genai.GenerativeModel('gemini-1.5-flash')`
With system message	`GenerativeModel(model_name=..., system_instruction=...)`

Configure output	`GenerationConfig(temperature=0.7, max_output_tokens=300)`
Normal call	`model.generate_content(prompt, generation_config=cfg)`
Streaming call	`model.generate_content(prompt, stream=True)`
Stop sequences	`GenerationConfig(stop_sequences=['###', 'END'])`
Count tokens	`model.count_tokens(text).total_tokens`
Get usage	`response.usage_metadata.prompt_token_count`
Finish reason	`response.candidates[0].finish_reason`
Iterate stream	`for chunk in response: chunk.text`

Section 14: Project Extensions & Real-World Ideas

From training exercise to your portfolio project

Extend ArYABot into a Full Product

Extension	What to Build	New Concepts
Add a REST API	Wrap ArYABot in FastAPI — users call /chat endpoint	`FastAPI, Pydantic, async/await, POST endpoint`
Add a Web UI	React frontend with streaming via SSE or WebSocket	`JavaScript EventSource, fetch, useState`
Add PDF support	Students upload syllabus PDF → bot answers from it	`PyMuPDF, file upload, context injection`
Multi-user sessions	Each student has own history, one shared cache	`dict of histories keyed by user_id, UUID`
Voice input	WhatsApp bot using Twilio → voice → text → ArYABot	`Twilio API, speech-to-text, webhook`
Quiz Generator	Given a topic, generate MCQ quiz as JSON	`Structured output + JSON schema enforcement`
Analytics Dashboard	Track which topics are asked most by students	`pandas, matplotlib, aggregation on session_log`
Deploy to Cloud	Host on Google Cloud Run or Oracle Cloud Free Tier	`Docker, environment variables, CI/CD, OCI`

FastAPI Extension — Quick Start:

```
# ── api_server.py ──────────────────────────────────────────────
# Wrap ArYABot in FastAPI — now it's a real web service!
# Install: pip install fastapi uvicorn
# Run:     uvicorn api_server:app --reload

from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from arya_bot import AryaBot
import asyncio

app = FastAPI(title='ArYABot API')
bot = AryaBot(budget_usd=5.0)   # shared bot instance

class ChatRequest(BaseModel):    # Pydantic: auto-validates request body
    message: str
    user_id: str = 'default'

# ── Non-streaming endpoint ─────────────────────────────────────
@app.post('/chat')
async def chat(req: ChatRequest):
    full_response = ''
    for chunk in bot.chat(req.message):
        full_response += chunk
    return {'response': full_response, 'stats': bot.stats}

# ── Streaming endpoint (like ChatGPT) ──────────────────────────
@app.post('/chat/stream')
async def chat_stream(req: ChatRequest):
    def generate():         # generator for StreamingResponse
        for chunk in bot.chat(req.message):
            yield f'data: {chunk}\n\n'    # SSE format
    return StreamingResponse(generate(), media_type='text/event-stream')

# ── Stats endpoint ─────────────────────────────────────────────
@app.get('/stats')
async def stats():
    return bot.stats

# Test with: curl -X POST http://localhost:8000/chat \
#   -H 'Content-Type: application/json' \
#   -d '{"message": "What is a decorator?"}'
```

Real World: *This is a production architecture. FastAPI + streaming response + AI backend is how Perplexity, You.com, and every AI search product works. Deploy this on Oracle Cloud Free Tier (you already have OCI certification!) and you have a live portfolio project.*

Summary: What You've Built & Learned

From 'what is temperature' to a full production AI system

Lectures 15 & 16 Covered:	Python Mastery Gained:

3.1 Temperature & Top-p — creativity dial
3.2 Max Tokens & Stop Sequences — length control
3.3 System Messages — AI personality
3.4 Streaming — word-by-word output
4.1 Token counting & @lru_cache
4.2 Chunking large documents
4.3 Cost tracking & budget enforcement
4.4 Prompt caching with hashlib
4.5 Rate limiting & @with_retry decorator
Structured JSON output mode
CAPSTONE: Full ArYABot system
	`class, __init__, @property, @dataclass`
	`yield (generators), for loop (iteration)`
	`@decorator (closures, @wraps)`
	`hashlib.md5, json.loads/dumps`
	`time.time(), time.sleep()`
	`re.sub(), re.search()`
	`try/except/else/raise`
	`TypedDict, list comprehension, lambda`
	`@lru_cache, functools.wraps`
	`enumerate(), zip(), range(), sum()`

Final Challenge — Ship It!

1. Get arya_bot.py working locally with your own GEMINI_API_KEY
2. Ask ArYABot 5 real questions from your syllabus and check answers
3. Add ONE new feature of your choice (PDF support, voice, quiz generator)
4. Wrap it in FastAPI (Section 14 starter code) and test with curl or Postman
5. **Push to GitHub. This IS your portfolio — future employers will see it.**